Graph Traversal Algorithms

A Comparison of Depth-First Search and Breadth-First Search

1. Introduction

A graph is a data structure consisting of a set of nodes, also called vertices, connected by edges. Graphs are used to model relationships between objects across many domains, including social networks, transportation systems, and computer networks. In this project, graphs are represented using an adjacency list, a dictionary in which each node maps to a list of its neighboring nodes. This representation is memory-efficient for sparse graphs and provides direct access to a node's neighbors in time proportional to the number of edges it has.

Graph traversal refers to the process of visiting every node in a graph in a systematic order, starting from a designated node. Traversal algorithms are fundamental to many graph-based problems, including pathfinding, cycle detection, and connected component analysis. This project implements and compares two classic graph traversal algorithms: Depth-First Search (DFS) and Breadth-First Search (BFS). Both algorithms are applied to randomly generated undirected, unweighted connected graphs of increasing size, and their execution times are measured and analyzed against their theoretical time complexities.

2. Algorithm Explanations

2.1 Depth-First Search (DFS)

Depth-First Search is a traversal algorithm that explores a graph by going as deep as possible along each branch before backtracking. It is implemented iteratively using an explicit stack. Beginning at the start node, the algorithm pushes the node onto the stack, marks it as visited, and then pushes each unvisited neighbor onto the stack. On each iteration, the top node is popped from the stack and its unvisited neighbors are pushed. This continues until the stack is empty, at which point all nodes reachable from the start node have been visited. The iterative implementation is used here instead of recursion to avoid Python's built-in recursion limit, which would cause failures on large graphs. The time complexity of DFS is O(V + E), where V is the number of vertices and E is the number of edges, because every node and every edge is visited exactly once. In the best case, the target node is found immediately at the start node. In the worst case, the algorithm must traverse the entire graph before finding the target or exhausting all reachable nodes. On average, DFS is expected to visit roughly half of the reachable nodes before terminating.

2.2 Breadth-First Search (BFS)

Breadth-First Search is a traversal algorithm that explores a graph level by level, visiting all nodes at the current depth before moving to nodes at the next depth. It is implemented using a queue, specifically a Python deque, for efficient first-in, first-out operations. Beginning at the start node, BFS marks it as visited and enqueues it. On each iteration, the front node is dequeued, and each unvisited neighbor is marked as visited and enqueued. Marking nodes as visited at enqueue time rather than dequeue time is a critical implementation detail that prevents the same node from being added to the queue multiple times. Like DFS, the time complexity of BFS is O(V + E), since every node and edge is processed exactly once. The best case occurs when the target node is adjacent to the start node and is found after examining only the first level. The worst case requires exploring the entire graph. On average, BFS is expected to find any given node faster than DFS when the target is close to the start node, due to its level-order exploration pattern.

3. Execution Time Results

Table 1 presents the average execution times for DFS and BFS across nine graph sizes, ranging from 10 to 5,000 nodes. For each graph size, a random connected graph was generated with a number of extra edges equal to the number of nodes, producing moderately dense graphs. Each algorithm was run three times per graph size and the average execution time was recorded using Python's time.perf_counter() for high-resolution timing.

Number of Nodes	DFS Avg Time (s)	BFS Avg Time (s)
10	0.000006	0.000004
50	0.000020	0.000015
100	0.000049	0.000031
250	0.000109	0.000068
500	0.000212	0.000149
750	0.000419	0.000337
1000	0.000827	0.000652
2500	0.001627	0.000910
5000	0.002939	0.002014

Table 1. Average execution times (seconds) for DFS and BFS across nine graph sizes. Each value is the average of three runs.

4. Time Complexity Analysis

Both DFS and BFS carry a theoretical time complexity of O(V + E). This means execution time should grow linearly with the number of nodes and edges in the graph. The results in Table 1 confirm this expectation. Across all nine graph sizes, execution times for both algorithms increase in a near-linear pattern. For example, DFS time

increases from 0.000006 seconds at 10 nodes to 0.002939 seconds at 5,000 nodes, a proportional increase consistent with a linear growth rate. BFS shows the same trend, increasing from 0.000004 seconds to 0.002014 seconds across the same range. The line chart in Figure 1 makes this linear trend visually clear for both algorithms.

Despite sharing the same asymptotic complexity, BFS consistently outperforms DFS in raw execution time across every graph size tested. At 5,000 nodes, DFS takes approximately 0.002939 seconds compared to BFS at 0.002014 seconds, meaning DFS is roughly 46% slower at that scale. This difference can be attributed to the distinct data structures each algorithm uses. DFS uses a Python list as a stack, where append and pop operations on the right end are O(1) but involve frequent reversals of neighbor lists to maintain consistent traversal order. BFS uses a collections.deque, which provides O(1) append and popleft operations with lower constant-factor overhead and better memory access patterns for sequential level-order exploration.

The consistent performance gap between DFS and BFS, visible in both the line and bar charts, does not contradict the theoretical O(V + E) complexity that both algorithms share. Asymptotic complexity describes the growth rate of an algorithm, not its absolute speed. Two algorithms can share the same Big-O class while having different constant factors. The data show that BFS has a smaller constant factor than DFS on this hardware and implementation, resulting in lower absolute execution times. Both algorithms scale linearly, which matches theoretical predictions.

5. Visualizations

Figure 1 shows the line chart of average execution times for DFS and BFS plotted against graph size. The near-linear upward trend of both lines confirms the O(V + E) theoretical complexity. BFS (orange) remains consistently below DFS (blue) across all graph sizes, reflecting the constant-factor performance advantage discussed in Section 4.

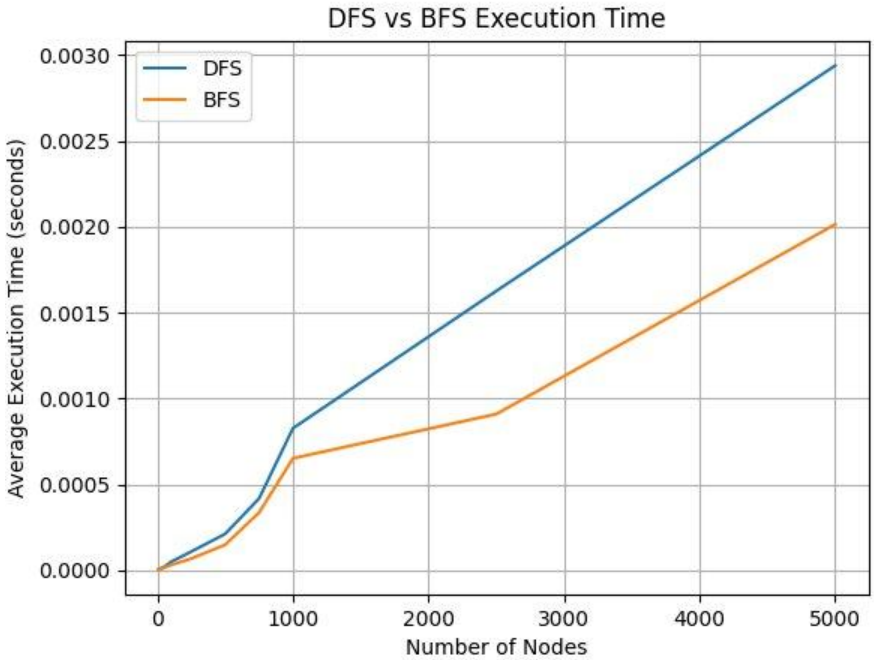

Figure 1. Line chart comparing average DFS and BFS execution times (seconds) vs. number of nodes.

Figure 2 shows the grouped bar chart of the same data. Each pair of bars represents one graph size, with DFS (blue) on the left and BFS (orange) on the right. The bar chart makes the absolute time difference between the two algorithms at each graph size more visually distinct than the line chart, particularly at larger node counts where the gap widens.

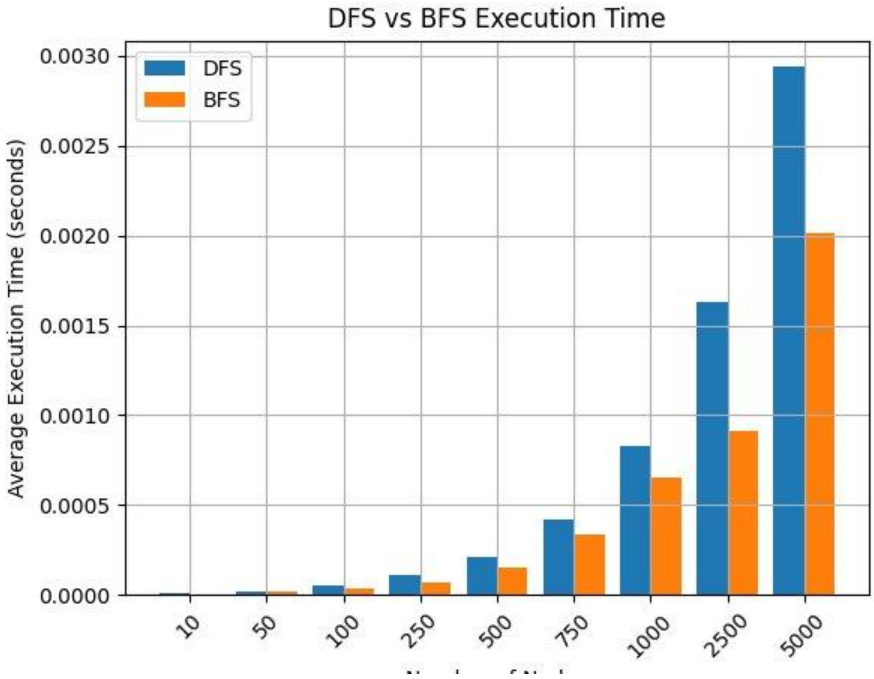

Figure 2. Grouped bar chart comparing average DFS and BFS execution times (seconds) vs. number of nodes.

6. Conclusions

This project implemented and compared Depth-First Search and Breadth-First Search on randomly generated undirected, unweighted connected graphs ranging from 10 to 5,000 nodes. Both algorithms produced execution times that grew linearly with graph size, confirming the theoretical O(V + E) time complexity for each. BFS consistently outperformed DFS in absolute execution time across all graph sizes, with the performance gap widening at larger scales, due to lower constant-factor overhead in the deque-based queue implementation compared to the list-based stack used by DFS. Future investigation could extend this comparison to include weighted graphs and Dijkstra's algorithm, examine space complexity by tracking peak stack and queue sizes during traversal, or test both algorithms on real-world graph datasets such as social or transportation networks to observe performance under irregular, non-random graph structures.